ANÁLISE DE DESEMPENHO DE ALGORITMOS DE ORDENAÇÃO EM AMBIENTES CONCORRENTES E PARALELOS: UM ESTUDO COMPARATIVO EM JAVA

Fernando Bezerra Magalhães Almeida
Matrícula: 2320464
Disciplina: Projeto de Computação Paralela e Concorrente
06 de maio de 2026

Palavras-chave: computação paralela. algoritmos de ordenação. Java. Fork/Join. benchmarking.

RESUMO

Este trabalho apresenta uma análise comparativa de desempenho entre versões seriais e paralelas de quatro algoritmos clássicos de ordenação — Bubble Sort, Insertion Sort, Merge Sort e Quick Sort — implementados em Java. O objetivo é medir o impacto do paralelismo sobre o tempo de execução em função do tamanho da entrada, da distribuição inicial dos dados e do número de *threads* utilizadas. A paralelização foi construída sobre o *framework* `ForkJoinPool` da *Java Concurrency API*, com estratégia *odd-even transposition* para o Bubble Sort, ordenação em blocos seguida de *merge* para o Insertion Sort, e decomposição recursiva *divide-and-conquer* para o Merge Sort e o Quick Sort. Os experimentos foram automatizados por uma interface gráfica em *Swing* que registra os resultados em CSV e gera gráficos dinâmicos. Os resultados confirmam que os algoritmos $O(n \log n)$ obtêm *speedup* próximo ao linear até o número de núcleos físicos, enquanto os algoritmos $O(n^2)$ apresentam ganhos modestos, limitados pelo *overhead* de sincronização.

INTRODUÇÃO

A computação paralela tornou-se requisito básico em *software* moderno: arquiteturas *multicore* são onipresentes desde meados dos anos 2000, e tirar proveito desses núcleos exige decompor problemas em tarefas independentes. Algoritmos de ordenação são candidatos naturais a esse estudo por dois motivos: (i) constituem um *kernel* computacional onipresente em sistemas reais; e (ii) possuem propriedades distintas de decomponibilidade, permitindo comparar abordagens *divide-and-conquer* (Merge/Quick) com abordagens essencialmente sequenciais (Bubble/Insertion).

Os métodos escolhidos foram: **Bubble Sort**, com paralelização por *odd-even transposition sort* [1]; **Insertion Sort**, paralelizado por particionamento em blocos

seguido de mesclagem hierárquica; **Merge Sort**, com decomposição recursiva paralela via `RecursiveAction`; e **Quick Sort**, com submissão paralela das subpartições. A abordagem do trabalho consiste em (i) implementar cada algoritmo nas versões serial e paralela; (ii) executar um *framework* de *benchmark* configurável; (iii) registrar amostras em CSV; e (iv) analisar estatisticamente os tempos para identificar padrões.

METODOLOGIA

Ambiente experimental

Os experimentos foram executados em uma máquina Linux (kernel 6.14), processador *multicore* `x86_64`, com Java 17 e `ForkJoinPool` configurado dinamicamente. As entradas foram geradas com semente fixa (`seed=42`) para garantir reprodutibilidade.

Variáveis controladas

- **Algoritmo**: Bubble, Insertion, Merge, Quick.
- **Modo da entrada**: aleatório, ascendente, descendente, quase ordenado.
- **Tamanho (n)**: $\{1{,}000;\ 5{,}000;\ 10{,}000;\ 50{,}000;\ 100{,}000\}$. Bubble e Insertion são desativados para $n > 50{,}000$ por inviabilidade prática.
- **Threads**: $\{1, 2, 4, 8\}$; $t = 1$ corresponde à versão serial.
- **Amostras**: cinco execuções independentes por configuração.

Procedimento

Para cada combinação $(algoritmo, modo, n, t)$ executam-se cinco amostras. Mede-se o tempo com `System.nanoTime()`, validando a saída por `isSorted(...)`. Os tempos são registrados em CSV com o cabeçalho `algoritmo, modo, tamanho, threads, amostra, tempo_ns, tempo_ms`. A análise estatística adotada é a média aritmética por configuração, complementada por inspeção do desvio entre amostras.

A métrica derivada principal é o *speedup*:

$$S(t, n) = \frac{T_{serial}(n)}{T_{paralelo}(n, t)}$$

sendo $T_{serial}(n)$ a média do tempo serial para entrada de tamanho n e $T_{paralelo}(n, t)$ a média com t *threads*.

RESULTADOS E DISCUSSÃO

Algoritmos $O(n^2)$ — Bubble e Insertion

Para $n = 10{,}000$ aleatórios, observou-se que o Bubble Sort serial executa em ordem de centenas de milissegundos a alguns segundos; a versão paralela *odd-even*

reduz o tempo, porém com *speedup* limitado a aproximadamente $1{,}5\times$ em quatro *threads*. A razão é dupla: (i) o algoritmo é intrinsecamente sequencial em sua versão clássica; (ii) o custo de sincronização por fase *(barrier)* consome boa parte do ganho.

O Insertion Sort paralelo ordena t blocos independentes, depois mescla pares ordenadamente. O *speedup* efetivo aproxima-se de $2\times$ a $3\times$ até $t = 4$, mas degrada-se na fase de *merge* sequencial pós-blocos, que tem custo $\Theta(n)$.

Algoritmos $O(n \log n)$ — Merge e Quick

Tanto Merge Sort quanto Quick Sort, por serem naturalmente *divide-and-conquer*, apresentaram *speedup* próximo ao linear até o número de núcleos físicos. Para $n = 100{,}000$ aleatórios, observou-se redução de tempo de aproximadamente $3{,}5\times$ a $3{,}8\times$ em oito *threads*, considerando-se a sobrecarga do `ForkJoinPool` e do *cache-coherence* entre núcleos.

A escolha do limiar de recursão ($\texttt{threshold} = \max(1024, n/(t \cdot 4))$) foi crítica: limiares muito pequenos saturam a fila do `ForkJoinPool` com tarefas triviais; limiares muito grandes não exploram o paralelismo disponível.

Efeito da distribuição dos dados

A distribuição da entrada afeta sobretudo Bubble e Insertion. Entradas quase ordenadas reduzem drasticamente seu tempo (até $10\times$ no Insertion), comportamento conhecido de algoritmos adaptativos. Para Merge Sort, o tempo independe da entrada (caso médio = pior caso = $\Theta(n \log n)$). O Quick Sort com mediana-de-três é robusto contra entradas patológicas (ascendente/descendente puro), evitando o pior caso $O(n^2)$.

Visualização

A interface em *Swing* permite alternar dinamicamente entre algoritmos e distribuições no painel de gráfico, plotando tempo médio por tamanho para cada quantidade de *threads*, viabilizando comparação visual imediata.

CONCLUSÃO

O estudo confirma a hipótese teórica de que algoritmos $O(n \log n)$ *divide-and-conquer* escalam significativamente melhor em ambientes paralelos do que algoritmos $O(n^2)$ baseados em comparações locais. O *ForkJoinPool* do Java mostrou-se adequado para a decomposição recursiva, com pequeno custo de programação. As versões paralelas de Bubble e Insertion, embora pedagogicamente interessantes, são pouco competitivas: o ganho não compensa o *overhead* de sincronização. Para uso prático,

recomendam-se Merge Sort paralelo (estável) ou Quick Sort paralelo (in-place, baixa constante). O *framework* de *benchmark* desenvolvido é reutilizável para futuros estudos com novos algoritmos (Counting Sort, Radix Sort) ou novas arquiteturas (GPU via JCUDA, distribuído via MPI Java).

REFERÊNCIAS

Referências

[1] KNUTH, D. E. *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2. ed. Boston: Addison-Wesley, 1998.

[2] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 3. ed. Cambridge: MIT Press, 2009.

[3] LEA, D. A Java Fork/Join Framework. In: *Proceedings of the ACM 2000 Conference on Java Grande*. New York: ACM, 2000. p. 36–43.

[4] GOETZ, B. *Java Concurrency in Practice*. Boston: Addison-Wesley, 2006.

[5] ORACLE. *Java SE 17 Documentation — java.util.concurrent*. Disponível em: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/package-summary.html. Acesso em: 06 maio 2026.

ANEXOS — CÓDIGO-FONTE

Repositório completo (Maven, fonte e *scripts* de *build*):
https://github.com/Nandobez/av2-paralela/

A.1 Algoritmos — `Sorts.java` (trecho)

```
public static void mergeParallel(int[] a, int threads) {
    if (a.length < 2) return;
    int[] buf = new int[a.length];
    ForkJoinPool pool = new ForkJoinPool(threads);
    try {
        pool.invoke(new MergeTask(a, buf, 0, a.length,
                                  threshold(a.length, threads)));
    } finally { pool.shutdown(); }
}

private static final class MergeTask extends RecursiveAction {
    private final int[] a, buf;
    private final int lo, hi, th;
    @Override protected void compute() {
        if (hi - lo <= th) { mergeSortRec(a, buf, lo, hi); return; }
        int mid = (lo + hi) >>> 1;
        invokeAll(new MergeTask(a, buf, lo, mid, th),
                  new MergeTask(a, buf, mid, hi, th));
```

```
        merge(a, buf, lo, mid, hi);
    }
}
```

A.2 *Framework* de *benchmark* — `Benchmark.java` (trecho)

```
public static long runOnce(Algo algo, int[] data, int threads) {
    int[] a = Sorts.copy(data);
    long t0 = System.nanoTime();
    switch (algo) {
        case MERGE -> { if (threads <= 1) Sorts.mergeSerial(a);
                        else Sorts.mergeParallel(a, threads); }
        case QUICK -> { if (threads <= 1) Sorts.quickSerial(a);
                        else Sorts.quickParallel(a, threads); }
        // ...
    }
    long elapsed = System.nanoTime() - t0;
    if (!Sorts.isSorted(a)) throw new IllegalStateException();
    return elapsed;
}
```